PGR111, solutions week 03

Task 1 - concepts and terminology

Briefly explain the following terminology and database concepts:

- Foreign key
- Primary Key
- The null value (“NULL”)
- Auto increment (“AUTO_INCREMENT”)

Write down the answers and save them. Might come in handy later in the semester? :-)

Task 2 - SQL

For the first 11 tasks, create a new MySQL sample database ("create schema"). Call it **people** and set it as "default schema". (Task 12+ uses World, as before.)

Write SQL against the new **people** database (schema) that performs the following:

1. Create a table called `person`. The table should contain:
   `ID, int`, autogenerated.
   `Ssn, char(11)`, should not be `null` and should be unique.
   `Name, varchar(100)`, should have default value `'Unknown'`.
   `Email, varchar(100)`, should be unique.
   Use the column `ID` as Primary Key.

   Check that the table is created by refreshing the object browser on the left in DBeaver or MySQL Workbench, then clicking *People* and its tables.

   ```
   CREATE TABLE person
   (
   ID INT AUTO_INCREMENT,
   Ssn CHAR(11) NOT NULL UNIQUE,
   Name VARCHAR(100) DEFAULT 'Unknown',
   Email VARCHAR(100) UNIQUE,
   PRIMARY KEY (ID)
   );
   ```

2. Enter a row in the person table. Specify the following data:
   Ssn: `'12345678901'`
   Name: `'Ola Nordmann'`
   Email: `'ola@nordmann.no'`

```
INSERT INTO person (Ssn, Name, Email) values
('12345678901', 'Ola Nordmann', 'ola@nordmann.no');
```

Write an appropriate query to see that the data is in the table.

```
SELECT * FROM person;
```

3. You decide that it's nice to record date of birth as well. Modify the existing `person` table, so that it also contains the column: `DateOfBirth`

```
ALTER TABLE person
ADD DateOfBirth DATE;
```

Check that the table has been updated to include the changes. (Same procedure as in task 1, or just write a query that displays all columns.)

As in task 1 above, or:

```
SELECT * FROM person;
```

4. Add a new row in the `person` table. Specify the following data:
   `Ssn: '98765432109'`

```
INSERT INTO person (Ssn) values
('98765432109');
```

Write an appropriate query to see that data resides in the table as expected.

```
SELECT * FROM person;
```

5. Update the row with the ssn `'98765432109'`, so that the date of birth for the selected person will be: `'1990-01-01'`. (**Note:** Date fields are stored in MySQL with year first, then month, then day, as shown here.)

```
UPDATE person
SET DateOfBirth = '1990-01-01'
WHERE Ssn = '98765432109';
```

Write an appropriate query to see that data is updated as expected.

```
SELECT * FROM person;
```

6. Add yourself in the person table! Fill in all fields (you don't need to use real info if you don't want to).

You have to have the solution for this yourself! :-P But the format is:

```
INSERT INTO PERSON (Ssn, Name, Email, DateOfBirth) VALUES
('<number>', '<name>', '<user>@student.kristiania.no', 'YYYY-MM-DD');
```

7. We realize that our database is a bit to restricted in its design: Each person can only have 1 email address. Create a separate table called `email` with the following criteria:
   `Address, varchar(75)`, should not be `null` and should be unique.
   `Type, varchar(50)`. *(Example content: 'work', 'school', 'private', etc.)*
   `Person_ID, int`, should not be able to be `null`.
   For the primary key, use the `Address` column.
   As a foreign key, use `Person_ID` that reference the `person` table's column `ID`.

   ```
   CREATE TABLE email
   (
   Address VARCHAR(75) NOT NULL UNIQUE,
   Type VARCHAR(50),
   Person_ID INT NOT NULL,
   PRIMARY KEY (Address),
   FOREIGN KEY (Person_ID) REFERENCES person (ID)
   );
   ```

   Check that the table has been updated to include the changes. (Same procedure as in task 1, or just write a query that displays all columns.)

   ```
   SELECT * FROM email;
   ```

8. Write an SQL query that copies email addresses and IDs from the Person table to the Address table and to the `Person_ID` column in the Email table. *(**Tip:** You need to feed the insert with a query. Replace "values (...)" with "select...". If you're not sure how to do it, look at chapter 3.3.1 in the Norwegian textbook, or look it up online.)*

   ```
   INSERT INTO email (Person_ID, Address)
   SELECT ID, email
   FROM person
   WHERE email IS NOT NULL;
   ```

9. (If you have done task 8.)
   Now that we have moved the email addresses to a separate table, we can delete the email column from the `person` table. *Note*: We want to remove the entire column, not just the data in it.

   ```
   ALTER TABLE person DROP COLUMN email;
   ```

10. Create a table like the one in task 1 but call it `personCopy`. (No need to copy data into it.) Check that the table is created. Then write the SQL command to delete it. *Note:* The idea is to test the command to delete a table, without destroying the two tables you've been working

on for a while now. :-)

create table as in task 1 (but with the name personCopy), then: Right click the left side of MySQL Workbench, select "Refresh All", to see that the table has been created.

To delete it, write:
DROP TABLE personCopy;

Check that the table has been deleted.

Right click the left side of MySQL Workbench, select "Refresh All", see that the table is gone again.

11. **DIFFICULT:** If we extract the name and date of birth from the person table, the date comes on MySQL's for us "backwards format". In a SELECT, reformat the output so date is shown on the format: 31-01-2001. (<-- Example of date with the correct format.)
Tip: *Read about the MySQL function: DATE_FORMAT() (w3schools or another online guide)*

(Note the mix of small and capital letters after the percentage signs. Must be like this!)
SELECT name, DATE_FORMAT(DateOfBirth, '%d-%m-%Y')
FROM person;

Add the date for Ola Nordmann. He should have DateOfBirth 20.02.1912 (old man!). Write it on this format ('dd.mm.yyyy'), use STR_TO_DATE() for MySQL to understand the format.

(Note the mix of small and capital letters after the percentage signs. Must be like this!)
UPDATE person
SET DateOfBirth = STR_TO_DATE('20.02.1912', '%d.%m.%Y')
WHERE name = 'Ola Nordmann';

Use the MySQL sample database ("schema") **world** for the remaining tasks.
(Set **world** as "default schema".)

Write SQL for the **world** database (schema) that performs the following:

12. Delete the row containing the city 'Bærum' from the city table. (This is no city and should not be in the city table!).

DELETE FROM city WHERE name = 'B%rum';
(We might get into trouble with the letter 'æ' due to character encoding. The wildcard usage above is a work-around for this. Ideally, we should have fixed the encoding, but it is as it is.)

Write an appropriate query to see that data is deleted as expected.

SELECT * FROM city WHERE name LIKE 'B%rum';

13. Use the max() function to retrieve the largest ID in world.city.

    SELECT MAX(id) FROM city; -- -> 4079

    SELECT * from information_schema.tables for the world database (WHERE table_schema = 'world'). Can you find out how many rows you have in your different tables? What is the name of the column indicating this? Can you find the column that gives us the next value for city.ID?

    SELECT * FROM information_schema.tables WHERE table_schema = 'world';
    TABLE_ROWS = 4035?! (← varies from one installation to another)
    Hmmm. Would think they should be the same? I'm asking Google for help. It looks like TABLE_ROWS isn't that accurate.
    I find the next value in the column: AUTO_INCREMENT (value 4080).

Tip: Besides asking the teacher assistants, lecturer and using the forum on Canvas, look at http://www.w3schools.com/sql/default.asp. Here you will find answers to many of your SQL questions :-) Also remember that the textbook has many nice resources, including answers to questions from the book. You will learn a lot from solving the puzzles in the book too!

You can also spend time exploring other databases from the textbook, or in DBeaver / MySQL Workbench / online. (Ex: the SAKILA database.)